



Generative AI solutions for software quality: Assessing industrial readiness

Andre Gheventer¹ · Patricia do Amaral Gurgel¹ · Carlos Henrique Brito² ·
Rafael Maiani de Mello¹ · Sabrina Rocha³ · Rodrigo Feitosa³ ·
Guilherme Horta Travassos³

Received: 22 December 2025 / Accepted: 4 April 2026
© The Author(s) 2026

Abstract

Software quality is undergoing a profound transformation, driven by state-of-the-art research on the application of emerging technologies in software development processes. Specifically, the use of generative Artificial Intelligence (AI) may represent an opportunity to advance the state of practice in this domain. This study aims to assess the industrial readiness and availability of Generative AI-based solutions for software quality, classifying them according to ISO/IEC 25010 attributes and SDLC phases. An empirical assessment of the state of practice was conducted, employing a Rapid Multivocal Literature Review (RMLR) protocol as a data collection instrument to screen evidence from academic databases (Scopus) and grey literature (Google, GitHub, PapersWithCode). We identified 24 potentially usable solutions. However, the analysis reveals a low technological maturity, with most solutions being academic prototypes hampered by fundamental technical limitations and adoption challenges. These include the “last mile problem” in translating research prototypes into reliable, production-ready tools; the “strategic adoption dilemma” forcing practitioners to trade off between proprietary lock-in and high open-source infrastructure costs; and the “scarcity of realistic public data,” which drives a generalization gap due to reliance on synthetic or leaked benchmarks. Generative AI in software quality remains an emerging but immature field, hampered by a critical reliability gap between academic prototypes and industrial needs. Advancing this domain requires moving beyond a narrow code-centric focus to address the quality of the AI systems themselves, expanding research across all SDLC phases and ISO 25010 attributes. We conclude with a roadmap advocating for contamination-free benchmarks, explainable architectures, and robust guidelines for real-world integration.

Keywords Generative artificial intelligence · Large language model · LLM · Software engineering

1 Introduction

Software Engineering (SE) has undergone profound transformations in recent decades, driven by emerging technologies, with a recent emphasis on Generative Artificial Intelligence (AI), whose potential to impact software quality processes significantly has been widely researched. Recent studies have described expectations of significant productivity gains across various stages of the Software Development Life Cycle (SDLC), driven by these new technologies. Specifically in software quality, there is growing adoption of Generative AI-based solutions for SE activities such as vulnerability detection (Kalouptsoglou et al., 2025, 2026), test generation and automation (Alagarsamy et al., 2025; Tang et al., 2024), defect localization (Qin et al., 2025; Xu et al., 2026), code verification and security (Fakhoury et al., 2024; Washizaki, 2024). These solutions have been recognized for their ability to reduce human intervention, accelerate project schedules, decrease manual effort, shorten development time, and consequently contribute to improving software quality (Banh et al., 2025; Hou et al., 2024; Raghi et al., 2024).

However, software quality is a multifaceted concept. To assess the potential effectiveness of GenAI in these solutions, it is important to define quality beyond subjective metrics and align it with standardized models that subdivide quality into specific characteristics, such as functional suitability, reliability, performance efficiency, security, and maintainability (ISO/IEC, 2011). This model type provides researchers and practitioners with a systematic approach to benchmarking and evaluating software products based on clearly defined criteria (Rafi et al., 2025).

Despite these potential advantages, adopting Generative AI-based solutions in SE, especially those based on large language models (LLMs), has proven complex and challenging. Among them, issues related to ethics, security, privacy, dependence on external vendors, and the potential biases inherent to the AI models stand out (Russo, 2024; Terragni et al., 2025). Moreover, the effective adoption of these solutions depends on development teams' ability to integrate these technologies into their workflows (Russo, 2024). Beyond these cross-cutting challenges, prior research has also reported solution-specific technical limitations affecting real-world adoption, including issues related to robustness across heterogeneous software projects, high rate of false positives, limited explainability of model outputs, dependence on public data quality, and the difficulties in producing developer-actionable results in practical environments (Garousi et al., 2025; Siavvas et al., 2024; Zhou et al., 2025).

These systemic issues are underscored by recent empirical observations regarding the friction of adopting these technologies in real-world settings. For instance, Travassos et al. (2025) highlights significant integration barriers, a heavy reliance on ad-hoc prompting, and the burden of extensive manual validation to ensure quality in critical domains. In light of this scenario, mapping and analyzing the evidence-based Generative AI solutions available for use, while observing their limitations, challenges, and gaps, becomes a relevant task to investigate whether such scarcity of robust tools is a systemic characteristic of the field. For this study, we operationalize the key concepts of the research question as follows:

- **Availability:** Public accessibility of the solution artifacts, such as source code, binaries, or access via internet-based platforms, combined with sufficient documentation to allow independent exploration, replication, and evaluation by third parties.

- **Industrial Readiness:** Degree of technological maturity and operational stability required for portability and use in production environments. This encompasses not only robustness and scalability but also the identification of potential limitations and risks that could hinder industrial adoption.

Given this context, this study aims to answer the following research question:

RQ1: What is the current state of industrial readiness and availability of Generative AI-based academic and commercial solutions to support software quality activities in Software Engineering?

To answer this question, we conducted an empirical assessment of the state of practice. We employed a Rapid Multivocal Literature Review (RMLR) protocol as a data collection instrument to integrate evidence from both scientific and grey literature. This approach focuses on identifying, coding, and analyzing Generative AI-based solutions that are currently supporting or have clear potential for practical application in software quality activities.

Finally, the concept of “Generative AI-based solution” adopted in this study is broad regarding artifact types (e.g., tools, frameworks, applications), but strictly defined in terms of the underlying technology. Our focus is on solutions capable of producing or synthesizing new content associated with software quality activities. Consequently, approaches relying exclusively on predictive or discriminative models (typically encoder-only architectures such as the BERT family) for classification tasks are excluded from this study. This distinction ensures that the evaluation addresses the specific limitations and maturity challenges inherent to generative technologies, such as non-determinism and hallucination risks, distinguishing them from established predictive paradigms.

This study makes the following key contributions to the field of software quality in SE:

- A critical assessment of 24 potentially usable Generative AI-based solutions for software quality, detailing industrial readiness patterns and maturity trends through a qualitative analysis of representative evidence.
- A diagnostic analysis of the current technological landscape, identifying systemic limitations and challenges such as low technological maturity, a lack of industrial validation, and significant adoption barriers.
- A conceptualization of the underlying causes for the observed academia-industry gap, including the “Last Mile Problem” in academic research and the “Strategic Adoption Dilemma” faced by practitioners.
- A proposal of a roadmap for future research aimed at bridging this gap, advocating for industrial case studies, the development of experimental standards and quality guidelines, and an expanded scope beyond code-level quality to transform academic prototypes into reliable, production-ready systems.

The remainder of this paper is structured as follows: initially, in Section 2, we analyze related work to position our contribution; then, Section 3 details the empirical methodology and data collection protocol applied to identify evidence-based solutions available for use; next, Section 4 presents the assessment results derived from the identified solutions;

Section 5 provides a discussion of the findings, interpreting their meaning and situating them within the broader research context; Section 6 discusses the threats to the validity of the study; finally, Section 7 concludes the paper by summarizing the implications of our work and presenting a proposal of a roadmap for future research.

2 Related work

Several studies have investigated the role of Generative AI in Software Engineering (SE), aiming to understand how researchers and practitioners evaluate and apply this technology. To effectively position our contribution, we analyze the most relevant prior investigations, highlighting their scope, key findings, and limitations in the context of industrial readiness. Table 1 summarizes this analysis.

Regarding general overviews, Hou et al. (2024) conducted a comprehensive Systematic Literature Review (SLR) covering 395 articles to categorize the application of Large Language Models (LLMs) across 85 SE tasks. While this work provides a broad academic landscape, its extensive scope inevitably dilutes the depth of analysis regarding software quality, focusing on research trends rather than operational maturity. Similarly, Zhang et al. (2024) unified perspectives from NLP and SE, offering a critical technical analysis of model architectures. Their work highlights important issues, such as the manipulation of benchmarks, but remains a theoretical synthesis that does not address the practical availability of solutions for production environments. From a sociotechnical perspective, Bazzan et al. (2024) performed a Multivocal Literature Review (MLR) to explore risks like hallucinations and bias. While essential for understanding ethical impacts, this study discusses risks as general phenomena and does not technically evaluate how specific existing solutions mitigate these risks in industrial workflows.

Other studies have narrowed their scope to specific sub-domains. Schmid et al. (2025) focused on software architecture, identifying trends towards decoder-only models but restricting the analysis to architectural tasks and academic experimentation. In the security domain, Zhou et al. (2025) provided a specialized SLR on vulnerability detection and repair. They identified critical algorithmic limitations, such as the lack of high-quality datasets and input granularity. While definitive for security, their work centers on model performance metrics rather than the broader operational readiness of quality assurance solutions. Finally, addressing the state of practice in testing, Garousi et al. (2025) conducted a Systematic Tool Review of 56 commercial AI-powered tools. Their empirical study provides valuable insights into features like self-healing; however, the scope is strictly confined to proprietary testing products, limiting the analysis of the underlying Generative AI models.

As Table 1 illustrates, the existing body of knowledge has predominantly adopted either a high-level theoretical perspective or a narrow domain-specific focus. A critical gap emerges from this analysis: the lack of a consolidated assessment of the industrial readiness of Generative AI specifically for Software Quality activities. Previous efforts have largely mapped *what is theoretically possible* or *what commercial testing tools claim to do*. This study addresses this gap by employing a multivocal data collection protocol to capture both academic prototypes and available tools, evaluating their maturity against the immediate needs of practitioners, such as infrastructure requirements and support mechanisms, thereby bridging the disconnect between theoretical research and industrial adoption.

Table 1 Comparative analysis of prior investigations on generative AI in software engineering

Study	Methodology	Scope / Focus	Data Sources	Key Distinction from Our Work
Hou et al. (2024)	Systematic Literature Review (SLR)	Comprehensive, state-of-the-art overview of all LLM applications across 85 SE tasks, with a focus on models, datasets, and techniques.	Academic Databases Only (IEEE, ACM, arXiv, etc.)	Provides a broad academic landscape. Our study narrows the scope to software quality and emphasizes the state of practice by including grey literature to identify industrially applicable solutions.
Bazzan et al. (2024)	Multivocal Literature Review (MLR)	High-level analysis of the broad impact of Generative AI in SE, covering fundamental aspects, use cases, benefits, and risks (e.g., plagiarism, bias).	Academic (Google Scholar, IEEE Xplore) and Grey Literature (Google Search).	Explores the general impact and risks of GenAI. We provide a granular, technical assessment of specific solutions available for deployment.
Zhang et al. (2024)	Survey	Unifies the perspectives of NLP and SE on language models for code, covering 70+ models and 40+ tasks. Focus on the evolution and taxonomy of code models.	Academic Databases from both the SE and NLP communities.	Focuses on a technical and theoretical synthesis of the research field. This study provides a practitioner-oriented assessment of the readiness and availability of solutions.
Garousi et al. (2025)	Systematic Tool Review (STR) + Empirical Study	Evaluation of AI-powered testing tools from industry, assessing their features, benefits, and limitations through a new empirical study.	Industry sources (tool websites, user reviews) and primary data from their own experiment.	Focuses specifically on testing tools and generates new empirical data. Our assessment covers a broader quality scope and correlates tools with scientific evidence.
Schmid et al. (2025)	Systematic Literature Review (SLR)	Highly specific review of LLM applications for software architecture tasks (e.g., design decisions, generation) and vice-versa.	Top-tier academic SE conferences and journals.	Focuses narrowly on the architecture domain. We target the software quality domain and assess tool maturity.
Zhou et al. (2025)	Systematic Literature Review (SLR)	Specialized review of LLMs for vulnerability detection and repair, identifying algorithmic limitations.	Academic Databases (SE, AI, Security venues).	Focuses vertically on security metrics. Our study assesses industrial readiness (operational stability and support) across broader quality activities.
This Study	<i>Empirical Assessment of State of Practice (using RMLR protocol)</i>	<i>Generative AI solutions available for practical use in software quality activities.</i>	<i>Academic (Scopus) and Grey Literature (Google, GitHub and PWC.)</i>	<i>Unique Focus: Explicitly assesses industrial readiness by mapping solutions with potential for immediate adoption, bridging the gap between academic research and industrial needs.</i>

3 Methodology

This study aligns with recent investigations in software quality that use systematic review protocols as a rigorous diagnostic tool to establish the state of the art before proposing technical advancements or guidelines (Ahmad and Kuo, 2025). However, given our specific

objective of assessing *industrial readiness* rather than relying solely on academic theory, we adapted this approach to a Rapid Multivocal Literature Review (RMLR) protocol.

Rapid Literature Reviews (RLRs) are an agile form of knowledge synthesis, optimized to deliver timely evidence for emerging fields. While traditional Systematic Literature Reviews (SLRs) emphasize exhaustive coverage, aiming to “*identify, evaluate and interpret all available research*” (Kitchenham, 2004), RLRs streamline the process to capture key trends and immediate applicability (Cartaxo et al., 2018; Hamel et al., 2021; Pizard et al., 2024). Given the rapid growth of Generative AI, a traditional SLR protocol risks obsolescence before data extraction is even complete. Thus, an RLR protocol represents a methodologically sound choice to establish a rigorous baseline of the state of practice. Although this approach may not cover all the steps of a full SLR, recent empirical assessments suggest that RLRs enable the identification of findings in Software Engineering (SE) comparable to those obtained through comprehensive SLRs, while providing more timely, actionable evidence for decision-making and connecting scientific evidence with practitioner needs (Pena et al., 2024; Pizard et al., 2024).

An RMLR extends the RLR framework by incorporating grey literature (GL) into its search process, including materials not published in scientific or peer-reviewed venues, and is often grounded in the practical experience and technical insights of professionals. The use of grey literature in SE is strongly associated with evidence-based methods and literature reviews in the field, typically encompassing code repositories, white papers, and technical documentation, which are commonly identified through Google Search (Garousi et al., 2020, 2016). As noted by these authors, while GL is vital for capturing practical insights in emerging fields with strong industry involvement, it introduces risks regarding quality and source bias. To address these challenges and ensure credibility, we employed a quality appraisal procedure that filtered sources based on:

1. *Authority and Credibility*: Prioritizing sources from verifiable organizations or recognized practitioners over anonymous or unverified content.
2. *Technical Evidence*: Selecting sources that provide concrete use cases, tool categorizations, or architectural details, rather than purely opinion-based or high-level marketing descriptions.

This GL appraisal process ensured that the final selection included only reliable, evidence-based solutions, sourced from both peer-reviewed publications and grey literature, explicitly excluding subjective or purely promotional material. To systematize the analysis of these diverse sources, this assessment implemented a structured protocol based on the Rapid Multivocal Literature Review methodology. By adhering to the meta-protocol defined by Rocha et al. (2025), we established a rigorous foundation enabled not just the identification but the critical assessment of Generative AI-based solutions available for industrial application. To operationalize this objective, the main research question was decomposed into ten specific subquestions, guiding the systematic extraction and technical characterization of the collected evidence:

- **RQ1a.** Where can these solutions be found?
- **RQ1b.** What Generative AI models are used by the solutions?
- **RQ1c.** How can these solutions be used?
- **RQ1d.** What technical and operational constraints should be considered when adopting these solutions?

Table 2 PICOC elements used for search string construction

Elements	Terms
Population	"Test*" OR "Quality" OR "Validation*" OR "Verification*" OR "Technical Debt" OR "Defect Detection" OR "Software Inspection" OR "Model Check*" OR "Debug*" OR "Code Review"
Intervention	"Generative AI" OR "Generative Artificial Intelligence" OR "Generative Model*" OR "Large Language Model*" OR "Language Model*" OR "Small Language Model*" OR "LLM*" OR "RAG" OR "Retrieval Augmented Generation" OR "Natural Language Processing" OR "NLP" OR "AI Agent" OR "AI Multi-Agent"
Comparison	N/A
Outcome	"Application*" OR "Technolog*" OR "Approach*" OR "Method*" OR "Tool*" OR "Framework*" OR "Solution*" OR "Strateg*" OR "Model*" OR "System*" OR "Platform*" OR "Technique*"
Context	"Software Engineering"

- **RQ1e.** What are the limitations and challenges faced by the identified solutions?
- **RQ1f.** What is the maturity level of these solutions?
- **RQ1g.** Is there technical documentation, support, or an active community that facilitates their use in academic and industrial contexts?
- **RQ1h.** What types of users or roles are supported by these solutions?
- **RQ1i.** What type of technical support is offered?
- **RQ1j.** Is there collaboration with industry to validate the proposed solutions through case studies or experiments?

These subquestions guided data collection and synthesis throughout the RMLR, enabling a more detailed analysis aligned with the interests of both the scientific community and industry. The construction of the search string in both the scientific and grey literature was based on the *PICOC* approach - Population, Intervention, Comparison, Outcome, and Context (Petticrew and Roberts, 2006), widely used in systematic reviews in the computing field. The elements of our PICOC framework are detailed in Table 2. In this study, the Comparison element was deemed inapplicable, as no direct comparisons between interventions were expected or required to answer the research questions.

To ensure complete transparency and facilitate reproducibility, all hyperlinks presented throughout this study, whether in the main text, footnotes, or references, were accessed and verified as active on December 20, 2025.

3.1 Inclusion and exclusion criteria

To ensure the relevance of the evidence selected for analysis, we defined explicit inclusion and exclusion criteria, as presented in Tables 3 and 4. These criteria are not intended to assess the individual quality of Generative AI-based solutions available. Instead, the goal is to map and interpret the current landscape using an analytical framework derived from the research subquestions (RQ1a-RQ1j). These questions cover dimensions such as usage patterns, access and availability, technical and operational constraints, maturity, documentation, support, and potential for industrial application. These dimensions guide the application of

Table 3 Inclusion criteria

IC	Criteria
IC1	The source must be in the context of software engineering.
IC2	The source must present a Generative AI-based solution ^a , applicable to software quality.
IC3	The source must report a primary study or an application case in the field.
IC4	The source must describe an evidence-based solution with potential for industrial use.
IC5	The source must have been published after December 2021 ^b .
IC6	The source must provide data to answer at least one of the research questions.

^a This study focuses on identifying and analyzing solutions based on Generative AI (e.g., GPT, Llama, Codex), capable of producing new artifacts. Approaches that rely exclusively on discriminative models (e.g., BERT, RoBERTa, and CodeBERT) are excluded because they represent a distinct technological paradigm, being typically subject to specific architectural constraints (e.g., input token limits) that would distort the comparative diagnosis of industrial readiness targeted by this analysis. ^b The initial cutoff date was defined based on the public release of ChatGPT in 2022. This marks the transition to the modern Generative AI era, characterized by the democratization of access through user-friendly graphical interfaces and IDE plugins. While earlier generative models existed, this period saw a surge in interest from researchers and practitioners due to the ease of use and reduced technical barriers for model interaction

Table 4 Exclusion criteria

EC	Criteria
EC1	The source does not meet at least one of the inclusion criteria (IC)
EC2	The source is duplicated
EC3	The source is not available for free access or through institutional access
EC4	The source is written in a language other than English, unless an official English translation is available
EC5	The source is a secondary study

thematic coding to the selected evidence, allowing the identification of patterns, trends, and gaps in the use of Generative AI for software quality.

3.2 Search Strategy and data sources

To identify relevant publications, we designed and executed a search strategy encompassing both scientific and grey literature. The specific sources, search strings, and execution process are detailed below.

Data sources For the scientific literature, we selected Scopus as the sole database for study identification, following the approach adopted by Baldassarre et al. (2021) and Cartaxo et al. (2018), who highlight Scopus as the largest database of peer-reviewed literature in Software Engineering (SE) and has been widely used in rapid reviews in this domain. To expand our grey literature search, we used Google Search with two domain-specific restrictions¹ to focus on relevant content from these platforms. We chose GitHub because it is widely used for sharing and maintaining open-source software projects, and PapersWithCode for organizing open-access academic articles related to machine learning.

Search strings The search strings, presented in Table 5, were systematically constructed based on the PICOC approach detailed previously in this section (see Table 2). A key

¹Added to grey literature search strings: site:github.com and site:paperswithcode.com.

Table 5 Search strings

Base	Strings
Scopus	TITLE-ABS-KEY (("Test*" OR "Quality" OR "Validation*" OR "Verification*" OR "Technical Debt" OR "Defect Detection" OR "Software Inspection" OR "Model Check*" OR "Debug*" OR "Code Review") AND ("Generative AI" OR "Generative Artificial Intelligence" OR "Generative Model*" OR "Large Language Model*" OR "Language Model*" OR "Small Language Model*" OR "LLM*" OR "RAG" OR "Retrieval Augmented Generation" OR "Natural Language Processing" OR "NLP" OR "AI Agent" OR "AI Multi-Agent") AND ("Application*" OR "Technolog*" OR "Approach*" OR "Method*" OR "Tool*" OR "Framework*" OR "Solution*" OR "Strateg*" OR "Model*" OR "System*" OR "Platform*" OR "Technique*") AND ("software engineering")) AND PUBYEAR > 2021 AND (EXCLUDE (DOCTYPE ^a , "cr")) AND (LIMIT-TO (LANGUAGE, "English"))
Grey Literature	
Common	("Test" OR "Quality" OR "Validation" OR "Verification" OR "Technical Debt" OR "Defect Detection" OR "Software Inspection" OR "Model Checking" OR "Debug" OR "Code Review") AND ("Generative AI" OR "Generative Artificial Intelligence" OR "Large Language Model" OR "LLM") AND ("Software Engineering") after:2021
Google	Add-site:youtube.com ^b -site:github.com -site:paperswithcode.com
GitHub	Add site:github.com
PapersWithCode	Add site:paperswithcode.com/paper ^c

^a(DOCTYPE, "cr") = Conference review. These are not documents or articles, but rather a cover explaining a specific event.^bVideos were excluded to prioritize written sources that facilitate structured data extraction and content analysis.^cThe term "paper" was added to the query, aiming to return only articles in the results

distinction was made between the Scopus string and the grey literature string. During initial pilot searches, we observed that Google's search engine, regardless of the access method (conventional interface or API), did not reliably process the full, complex Boolean query used for Scopus. This often resulted in truncated results and significant loss of evidence, especially when filtering for specific platforms such as GitHub and PapersWithCode. This limitation is inherent to the search engine's functionality and is likely related to an internal cap on the number of logical connectors or search terms that can be included in a single query. To ensure query compatibility while preserving the core search intent, the grey literature string was reformulated by removing the 'Outcome' element from the PICOC structure.

Execution and refinement The search strings were executed on their respective platforms. For the scientific literature, the query was run on the Scopus advanced search interface, accessed via the institutional subscription provided by Brazil's CAPES Periodicals Portal². For the grey literature, the execution was multi-staged. First, the search string was tested on Google's standard search interface to validate the query and ensure the results were compatible and relevant. Once validated, the Google Custom Search API (Google, 2025) was used to extract results into a spreadsheet for management and analysis systematically. In line with standard practices for multivocal reviews (Cartaxo et al., 2018), we relied on Google Search's ranking algorithm and limited our inspection to the first 100 results for each grey literature query. Furthermore, we adopted an *adapted snowballing strategy* tailored for grey

² <https://www.periodicos.capes.gov.br/>

literature: during the review of the retrieved pages, we also explored relevant hyperlinks that pointed to other potentially pertinent solutions not captured by the initial search. This additional step aimed to broaden coverage and capture solutions that, although not directly listed among the top results of the queries, could be relevant to the review's inclusion criteria.

3.3 Execution and screening process

The search strings were applied between April and May 2025. They yielded 732 publications, of which 522 were retrieved from the Scopus database and 210 from Google Search, including 100 from Google itself, 100 from GitHub, and 10 from PapersWithCode. Although the plan was to evaluate the first 100 results from each platform, only ten results on PapersWithCode matched the search string. To ensure a rigorous and transparent selection of the most relevant studies, we conducted a multi-stage screening process, as illustrated in Fig. 1. The process was divided into two parallel streams, one for scientific literature and the other for grey literature.

The study selection process was conducted by three authors affiliated with the Postgraduate Program in Computer Science. The two primary reviewers work as Information Security Analysts in their respective organizations, while disagreements were resolved by a third author with solid industry background as a Software Developer. To ensure methodological rigor and consistency, the entire process was supervised by two other authors, both senior researchers with extensive experience in Evidence-Based Software Engineering

To quantitatively validate the consistency of the selection process, we established a statistical agreement protocol. We utilized the *Linear Weighted Cohen's Kappa* metric, which is suitable

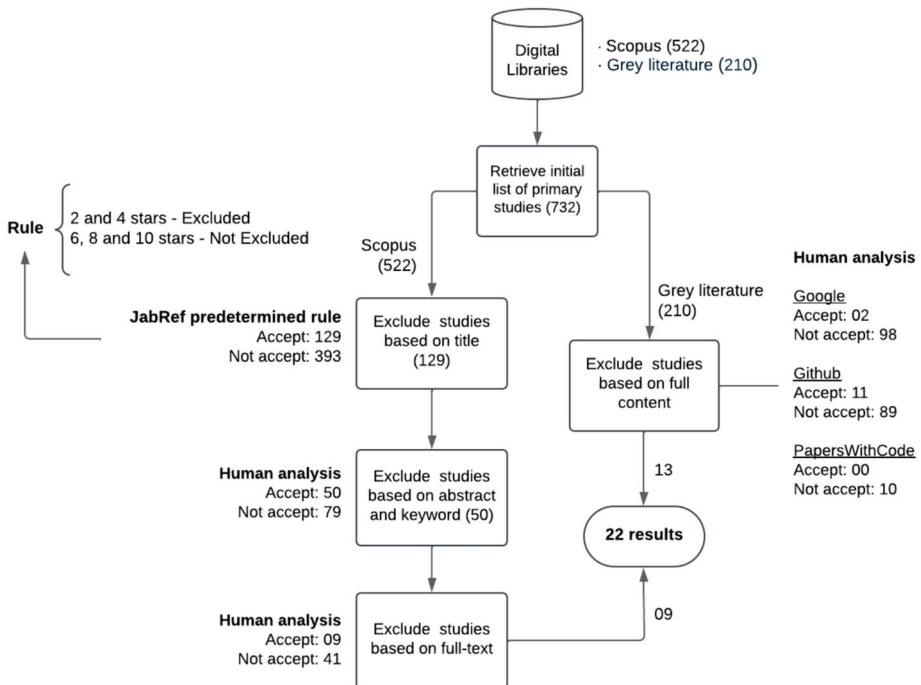


Fig. 1 RMLR screening process

Table 6 Kappa agreement interpretation scale (Landis and Koch, 1977)

Kappa Value	Strength of Agreement
< 0.00	Poor
0.00 – 0.20	Slight
0.21 – 0.40	Fair
0.41 – 0.60	Moderate
0.61 – 0.80	Substantial
0.81 – 1.00	Almost Perfect

for the ordinal nature of our scoring scale (1 = Not Accept < 3 = Doubt < 5 = Accept). Unlike simple agreement, this metric assigns partial credit to close matches while penalizing severe disagreements more heavily. To interpret the coefficients, we adopted the standard guidelines established by Landis and Koch (1977), as detailed in Table 6.

3.3.1 Screening of scientific literature

The 522 studies retrieved from Scopus underwent a three-stage screening process executed by two primary reviewers, with a third reviewer acting as a mediator. To ensure consistency and transparency, the reviewers applied a standardized rating scale strictly mapped to the inclusion (Table 3) and exclusion (Table 4) criteria. This systematic approach prioritized evidence retention over immediate exclusion. Specifically:

- 5 Stars (Accept): Assigned when the study appeared to satisfy all inclusion criteria (IC1–IC6), demonstrating clear relevance to Generative AI in software quality.
- 3 Stars (Doubt): Assigned when relevance could not be determined due to insufficient detail (e.g., vague title regarding IC4, ensuring that ambiguous cases were not discarded prematurely).
- 1 Star (Not accept): Assigned when the study clearly violated an inclusion criterion or met an exclusion criterion (e.g., EC5 - secondary study, or failing IC2 by using discriminative models).

Based on this classification scheme, the screening process was executed in three sequential stages:

1. **Stage 1 (Title Screening):** First, the metadata of all 522 publications were imported into two separate JabRef³ libraries, allowing two reviewers to assess them independently. Each reviewer classified the titles using a three-point scale: five stars (accept), three stars (doubt), and one star (reject). The inter-rater agreement for this stage yielded a Linear Weighted Kappa of 0.45 (Moderate Agreement). Given this level of agreement, which is consistent with the ambiguity of titles in emerging fields, we applied an inclusive consensus rule to prioritize recall: studies with a combined score of 6, 8, or 10 stars proceeded to the next stage. This strategy ensured that, alongside the 49 unanimously accepted studies, 80 studies with conflicting or doubtful ratings were retained for further examination. Conversely, 393 studies (score of 2 or 4) were excluded. The complete screening logic is detailed in Table 7. This initial screening reduced the candidate set to 129.

³ <https://www.jabref.org/>

Table 7 Screening logic and decisions for Stage 1 and Stage 2

Reviewer 1	Reviewer 2	Score	Stage 1 Decision	Stage 2 Decision
5 (Accept)	5 (Accept)	10	Proceeded	Included (Unanimous)
5 (Accept)	3 (Doubt)	8	Proceeded	Mediated (Reviewer 3)
3 (Doubt)	5 (Accept)	8	Proceeded	Mediated (Reviewer 3)
5 (Accept)	1 (Reject)	6	Proceeded	Mediated (Reviewer 3)
1 (Reject)	5 (Accept)	6	Proceeded	Mediated (Reviewer 3)
3 (Doubt)	3 (Doubt)	6	Proceeded	Mediated (Reviewer 3)
3 (Doubt)	1 (Reject)	4	Excluded	Mediated (Reviewer 3)
1 (Reject)	3 (Doubt)	4	Excluded	Mediated (Reviewer 3)
1 (Reject)	1 (Reject)	2	Excluded	Excluded (Unanimous)

- Stage 2 (Abstract and Keyword Screening):** The same two reviewers independently assessed the remaining 129 studies based on their abstracts and keywords. At this stage, the JabRef application was also used to assign the same classifications: 5, 3, or 1 stars. The inter-rater agreement yielded a Linear Weighted Kappa of 0.53 (Moderate Agreement). This improvement (compared to 0.45 in Stage 1) reflects the reduced ambiguity when accessing abstract-level details. In this stage, we applied a strict consensus mechanism: 30 studies unanimously accepted (5 and 5 stars) were automatically included, while 38 studies unanimously rejected (1 and 1 stars) were excluded. The remaining 61 cases, involving disagreements or doubts, were arbitrated by the third reviewer. After detailed analysis, the mediator selected 20 of these conflicting studies for inclusion. This stage refined the selection to 50 studies.
- Stage 3 (Full-text Analysis):** The final 50 studies were organized in a spreadsheet for the full-text review. These studies underwent a cross-review process where each paper was evaluated by two independent reviewers, retaining the 3-point scale to capture decision nuances. The average inter-rater agreement yielded a Linear Weighted Kappa of 0.70 (Substantial Agreement), confirming the reliability of the selection when full details were available. In this process, 7 studies were unanimously accepted (score 10), and 30 were unanimously rejected (score 2). The remaining 13 discrepancies were discussed in a consensus meeting involving the supervisors. This resolution process led to the inclusion of 2 additional studies, resulting in a final set of **9 primary studies** from the scientific literature.

3.3.2 Screening of grey literature

A similar structured approach was applied to the 210 grey literature sources retrieved. To manage the process, the titles and URLs of these sources were imported into a spreadsheet. Given the heterogeneous nature of this literature, we proceeded directly to a full-content review. The 210 sources were distributed among the three reviewers for an initial independent assessment, during which each was classified as "accepted," "rejected," or "doubtful." Following this initial classification, a cross-review stage was conducted, in which a second independent reviewer re-evaluated each source to ensure reliability. As with the scientific literature, any discrepancies between the two reviewers were resolved at a final consensus meeting attended by all reviewers. This rigorous process resulted in the selection of *13 primary sources* from the grey literature.

3.3.3 Final set of selected studies

Upon completion of both screening streams, a total of 22 *publications* (9 from Scopus and 13 from grey literature) were selected for data extraction and analysis. It is worth noting that while the initial search yielded a high volume of results, the strict application of Inclusion Criterion IC4 (evidence-based solution with potential for industrial use) significantly reduced the final set. This reduction, in itself, is a preliminary finding that underscores the scarcity of "ready-to-use" solutions amid a plethora of theoretical proposals in the field.

3.4 Data coding and synthesis

Data extraction was performed using a structured schema derived directly from the research questions (RQ1a-RQ1j). To ensure consistency and mitigate individual bias, the primary metadata and qualitative data were extracted by one of the authors and subsequently verified by a second independent reviewer. Following extraction, the analysis was conducted through an iterative process of *thematic coding* (Yang, 2025). This approach allowed themes, patterns, and categorizations to emerge directly from a deep reading and interpretation of the publications. Due to the heterogeneity of the data, we used this analysis to identify recurring patterns of readiness across the selected studies, highlighting representative examples that illustrate broader industrial trends. By transforming the raw descriptive content into structured analytical insights, we sought to gain a deeper understanding of the evidence contained in the collected material (Cruzes and Dyba, 2011).

We applied this method to create a structured overview of the mapped Generative AI-based solutions. This framework was guided by the dimensions outlined in our research sub-questions. A key step was classifying the *primary objective* of each solution, as presented in Tables 8 and 9. To this end, we analyzed the descriptive excerpts from each source. Since Generative AI tools are often multi-purpose, we established a *prioritization rule*: functionalities directly related to software quality were prioritized over generic capabilities. For example, if a solution offered both generic code generation and specific test case generation, it was coded under the latter to align with the scope of this work.

Following this prioritization, each solution was systematically categorized according to a taxonomy based on the ISO/IEC 25010 quality model and the Software Development Life Cycle (SDLC) phases. While these frameworks encompass a broader range of attributes and stages, our mapping focuses on the categories actually identified within the selected solutions:

- **Security:** Solutions focused on vulnerability detection, penetration testing, or secure code enforcement.
- **Reliability:** Solutions aiming to detect defects, bugs, or anomalies to ensure operational stability.
- **Maintainability:** Solutions that facilitate modification and testing, including automated test generation and instrumentation.

Additionally, solutions were mapped to their primary operating phase identified in our sample: Coding, Testing, or Operations. The resulting classification is detailed in 'Attr' and 'Phs' columns of Tables 8 and 9.

4 Results

This section presents the results of our technical assessment. Tables 8 and 9 categorize the 24 identified solutions⁴ based on their origin (academic prototypes vs. commercial products). The subsequent subsections evaluate their characteristics and maturity, structured according to our research questions (RQ1a-RQ1j).

The overall analysis reveals a heterogeneous landscape. The academic solutions, detailed in Table 8, are largely experimental prototypes with significant barriers to industrial adoption. In contrast, the commercial solutions listed in Table 9, already available on the market in software-as-a-service (SaaS⁵) format, demonstrate higher readiness for practical use. Although relevant as a contextual reference, these solutions were not considered in the formulation of research question answers, as they lack publicly accessible, specific technical details about their Generative AI implementations needed to answer our research questions (e.g., the particular models used, technical constraints, or datasets for training), making analyses using the same criteria applied to the academic solutions unfeasible.

An analysis of the Tables 8 and 9 reveals a distinct concentration of efforts within specific domains. Regarding quality attributes, the identified solutions predominantly target *Security* and *Reliability*, with a secondary focus on *Maintainability* through test automation. Other ISO 25010 characteristics, such as *Functional suitability* and *Performance efficiency*, were not prominently identified in our sample. In terms of the SDLC, the tools are clustered in the *Coding* and *Testing* phases. It is noteworthy that our mapping identified a complete absence of solutions addressing earlier phases such as *Requirements* and *Design*, demonstrating that current Generative AI solutions for software quality are heavily concentrated in downstream implementation and verification activities.

Research Question 1a

Where can these solutions be found?

The availability of developed solutions for replication and deployment purposes is not yet a consolidated practice. We observed that in two cases involving complex Retrieval-Augmented Generation (RAG) architectures, the necessary artifacts were not shared (De Vito et al., 2023; Sheng et al., 2024). However, many solutions have made their artifacts publicly accessible through various platforms. The most common platform is *GitHub*, which hosts the artifacts for several analyzed solutions (Ahmed et al., 2025; Bhandari et al., 2025; Kang et al., 2023, 2024; Kessel, 2025; Li et al., 2023; Mansur et al., 2024; Patel et al., 2024; Shao et al., 2025; Xia et al., 2024; Yu et al., 2025). Other platforms, such as *Figshare*⁶ (Steenhoek et al., 2025) and *Hugging Face*⁷ (Ahmed et al., 2025), were also used to facilitate reproducibility and adoption of these solutions. Complementarily, prompt engineering techniques are frequently documented directly within the technical reports (Chen et al., 2024; Kikuta et al., 2025; Phan et al., 2024; Yang et al., 2024). ⁶<https://figshare.com/> ⁷ <https://huggingface.co/>

⁴One of the selected primary sources presented three solutions.

⁵A service delivery model in which the user accesses applications via the internet, through paid or free (usually limited) subscriptions, without needing to perform local installations. https://csrc.nist.gov/glossary/term/software_as_a_service/.

⁶<https://figshare.com/>

⁷<https://huggingface.co/>

Table 8 Academic studies (non-commercial solutions)

ID	Study	Source	Type	Attr	Phs	Solution Objective ^a
1	De Vito et al. (2023)	Scopus	CP	REL	COD	Defect detection in source code and Test case generation
2	Happe and Cito (2023)	GitHub	CP	SEC	TST	Security penetration testing
3	Kang et al. (2023)	Scopus	CP	MNT	TST	Test case generation
4	Li et al. (2023)	Scopus	CP	REL	COD	Defect detection in source code and Test case generation
5	Chen et al. (2024)	GitHub	CP	REL	COD	Defect detection in source code
6	Kang et al. (2024)	Scopus	JA	MNT	TST	Test case generation (evolution of solution Kang et al. (2023))
7	Mansur et al. (2024)	Scopus	CP	REL	COD	Defect detection in source code
8	Patel et al. (2024)	Scopus	JA	MNT	TST	Test database generation
9	Phan et al. (2024)	GitHub	PR	REL	COD	Defect detection in source code
10	Sheng et al. (2024)	Scopus	CP	SEC	COD	Detection of security vulnerabilities in source code
11	Xia et al. (2024)	Scopus	CP	REL	COD	Defect detection in source code
12	Yang et al. (2024)	GitHub	CP	REL	COD	Defect detection in source code
13	Ahmed et al. (2025)	GitHub	PR	SEC	COD	Detection of security vulnerabilities in source code
14	Bhandari et al. (2025)	Scopus	JA	SEC	COD	Detection of security vulnerabilities in source code
15	Kessel (2025)	GitHub	PR	MNT	TST	Instrumentation for experiments and code property analysis
16	Kikuta et al. (2025)	GitHub	PR	REL	OPS	Chaos Engineering (CE) automation
17	Shao et al. (2025)	GitHub	PR	SEC	COD	Detection of security vulnerabilities in source code (various artifacts)
18	Steenhoek et al. (2025)	GitHub	PR	SEC	COD	Detection of security vulnerabilities in source code
19	Xia et al. (2025)	GitHub	JA	REL	COD	Defect detection in source code
20	Yu et al. (2025)	GitHub	CP	REL	COD	Defect detection in source code

Type: CP-Conference Paper; JA-Journal Article; PR-Preprint Article Attr (ISO 25010): SEC-Security; REL-Reliability; MNT-Maintainability Phs (SDLC Phase): COD-Coding; TST-Testing; OPS-Operation/Maintenance ^aIf the solution has multiple objectives, only those related to software quality were classified.

Table 9 Web references (commercial solutions)

ID	Industry Publications	Source	Name	Attr	Phs	Solution Objective
1	Index.dev (2024)	Google	GitHub Copilot ^a	REL	COD	Defect detection in source code and Test case generation
			Snyk platform ^b	SEC	COD	Detection of security vulnerabilities in source code
2	SEDaily (2024)	Google	Testim ^c	MNT	TST	Test automation
			GitLab Duo ^d	SEC	COD	Security vulnerability detection

Attr (ISO 25010): SEC-Security; REL-Reliability; MNT-Maintainability. Phs (SDLC Phase): COD-Coding; TST-Testing ^a<https://github.com/copilot/>^b<https://snyk.io/>^c<https://www.testim.io/>^d<https://about.gitlab.com/gitlab-duo/>

Research Question 1b

What Generative AI models are used by the solutions?

For analysis purposes, the Generative Artificial Intelligence models employed in the solutions were classified into two categories, as detailed below. This classification enables discussion not only of the technical aspects but also of the practical implications for adoption, such as infrastructure dependencies, data privacy, scalability, and operational costs.

- **Closed Models:** Generative AI models whose access is provided exclusively through interfaces and/or infrastructures controlled by their developers, including those made available freely through the web, with no possibility of local execution, direct access to the source code, or training components (weights).
- **Open Models:** Generative AI models that allow for local execution, direct access to the source code, and/or training components (weights).

Table 10 presents this classification for each of the analyzed solutions, while Table 11 highlights a subset of these models that are specifically trained on programming data.

Table 10 Classification of generative AI models used in the identified solutions

Study	Closed Models	Open Models
De Vito et al. (2023)	GPT-3.5	none
Happe and Cito (2023)	GPT-3.5	none
Kang et al. (2023)	Codex	none
Li et al. (2023)	GPT-3.5	none
Chen et al. (2024)	GPT-3.5, GPT-4, Codex	StarCoder
Kang et al. (2024)	GPT-3.5, Codex	StarCoder, Bloom, InCoder, CodeGen
Mansur et al. (2024)	none	Llama3
Patel et al. (2024)	GPT-3.5	Ollama ^a
Phan et al. (2024)	GPT-3.5	Llama3.1, WizardLM2, Code Llama, StarCoder
Sheng et al. (2024)	GPT-4o	none
Xia et al. (2024)	GPT-4	StarCoder
Yang et al. (2024)	GPT-4o, Claude-3-Opus	none
Ahmed et al. (2025)	GPT-4.1, Claude-3.7-Sonnet	Qwen2.5-Coder, Deepseek-Coder, Codestral
Bhandari et al. (2025)	none	CodeT5, Code Llama
Kessel (2025)	GPT-4o-mini	none
Kikuta et al. (2025)	Gpt-4o, Claude-3.5- Sonnet, Gemini-1.5-Pro	none
Shao et al. (2025)	GPT-4o, GPT-4.1, Claude-3.5-Sonnet, Claude-3.7-Sonnet, DeepSeek V3	none
Steenhoek et al. (2025)	GPT-3.5, GPT-4, Gemini-1.0-Pro	Mixtral-MoE, Code Llama, Llama2, WizardCoder, DeepSeek-Coder, StarChat, StarCoder, MagiCoder, Mistral
Xia et al. (2025)	GPT-4o	none
Yu et al. (2025)	Claude-3.5-Sonnet	none

^a It mentions that it can be used in models that run on Ollama via REST APIs

Table 11 Code-centric generative AI models identified

Model	Study	URL
StarCoder	Xia et al. (2024); Kang et al. (2024); Chen et al. (2024); Steenhoek et al. (2025); Phan et al. (2024)	https://github.com/bigcode-project/star-coder2/
Bloom	Kang et al. (2024)	https://huggingface.co/docs/transformers/model_doc/bloom/
InCoder	Kang et al. (2024)	https://sites.google.com/view/incoder-code-models/
CodeGen	Kang et al. (2024)	https://github.com/salesforce/codegen/
Codex	Kang et al. (2023, 2024); Chen et al. (2024)	https://openai.com/index/introducing-codex/
Code Llama	Steenhoek et al. (2025); Phan et al. (2024); Bhandari et al. (2025)	https://ai.meta.com/blog/code-llama-large-language-model-coding/
WizardCoder	Steenhoek et al. (2025)	https://huggingface.co/WizardLMTeam/WizardCoder-15B-V1.0/
DeepSeek-Coder	Steenhoek et al. (2025); Ahmed et al. (2025)	https://deepseek-coder.github.io/
MagiCoder	Steenhoek et al. (2025)	https://github.com/ise-uiuc/magicoder/
Qwen2.5-Coder	Ahmed et al. (2025)	https://ollama.com/library/qwen2.5-coder/
Codestral	Ahmed et al. (2025)	https://mistral.ai/news/codestral/
CodeT5	Bhandari et al. (2025)	https://github.com/salesforce/CodeT5/

Among the solutions analyzed, those that employ Generative AI models trained on intensive programming data stand out, as shown in Table 11. Although these models are not exclusive to SE, they were developed with a strong focus on source-code-related tasks, such as code generation, completion, review, and explanation. This specialization represents a substantial advance in adapting Generative AI to the domain's specific needs, thereby enhancing the effectiveness and applicability of the proposed solutions.

Research Question 1c

How can these solutions be used?

Our analysis shows that the investigated solutions are generally not made available in directly executable formats, such as installers or ready-to-use packages. Instead, they are provided through several distinct modalities, each with specific usage requirements:

- **Complex Architectures:** Solutions based on Retrieval-Augmented Generation (RAG) architectures require complex configurations involving the integration of various components, such as vector databases, mechanisms for generating embeddings, and a coordinated set of code components (De Vito et al., 2023; Mansur et al., 2024; Phan et al., 2024; Shao et al., 2025; Sheng et al., 2024).
- **Repository-Based Solutions:** Most solutions are provided as code repositories, typically on GitHub. While they include installation and configuration instructions, they still demand specialized technical knowledge for replication, including specific programming environments such as Python (Li et al., 2023; Phan et al., 2024; Xia et al., 2025, 2024) and Java (Kessel, 2025; Patel et al., 2024).
- **Containerized Proposals:** To simplify deployment, several proposals facilitate replication by providing Docker images (Kang et al., 2023, 2024; Kessel, 2025; Kikuta et al., 2025; Shao et al., 2025; Xia et al., 2024; Yu et al., 2025), which streamline the environment setup process using containers.
- **Portable Prompt Engineering Techniques:** A subset of solutions offers high portability by making prompt engineering techniques available. These typically consist of structured sets of prompts that can be adapted and applied to other contexts without requiring extensive supporting infrastructure (Chen et al., 2024; Kikuta et al., 2025; Phan et al., 2024; Yang et al., 2024).

Research Question 1d

What technical and operational constraints should be considered when adopting these solutions?

The analyzed solutions impose several technical and operational constraints, primarily related to computational resources, which vary significantly depending on the model type used: Closed Models or Open Models. A recurring constraint for solutions that execute Open Models locally is the need for robust computational infrastructure. This is particularly evident in the requirement for dedicated, high-performance GPUs from NVIDIA, such as the RTX A6000 (Xia et al., 2024), NVIDIA DGX / RTX 3080 (Bhandari et al., 2025), NVIDIA RTX 3090 (Kang et al., 2024), and NVIDIA L40S (Ahmed et al., 2025). No solutions were found to mention the use of GPUs from other manufacturers.

On the other hand, solutions based on Closed Models accessed via cloud APIs do not demand high-performance local resources. The primary constraint for this approach shifts from hardware to operational factors, such as reliance on external providers, API availability, and potential costs of use. An alternative approach was identified for executing Open Models locally with lower hardware requirements, using the Ollama platform⁸ (Patel et al., 2024). This platform supports local inference on CPUs, significantly lowering the hardware barrier. However, this approach introduces constraints: it delivers considerably lower performance than running on dedicated GPUs and may still impose limitations on latency and scalability for more complex tasks.

Research Question 1e

What are the limitations and challenges faced by the identified solutions?

⁸<https://ollama.com/>

Our analysis revealed several cross-cutting limitations and challenges that impact the industrial applicability and long-term stability of the evaluated solutions. Beyond operational constraints, we identified critical hurdles regarding generalizability, result precision, and semantic validity. We categorize these into fundamental technical limitations inherent in the solutions and practical difficulties associated with their adoption.

To characterize the evaluation landscape, table 12 summarizes the technical characteristics of the benchmarks employed by the primary studies, ordered chronologically. We identified a clear dichotomy in evaluation complexity. While earlier datasets like *QuixBugs* (Lin et al., 2017) and *Defects4J* (Just et al., 2014) focus on isolated, function-level defects, the recent emergence of *SWE-bench* (Jimenez et al., 2024) represents a significant shift towards increased industrial realism. By requiring models to resolve issues within full repository contexts, involving navigation across multiple files and dependency management, *SWE-bench* provides a more accurate proxy for the complexity of real-world software engineering tasks compared to earlier benchmarks. However, despite these advancements, our analysis identified critical hurdles stemming from data dependency, model opacity, and architectural constraints, as detailed below.

Table 12 Benchmarks datasets used by the evaluated solutions

Dataset	Authors	Languages	Technical Details
[1] Defects4J	Just et al. (2014)	Java	835 real-world bugs from 17 projects, including both failing and passing test cases for each defect.
[2] QuixBugs	Lin et al. (2017)	Python, Java	40 programs from the "Quixey Challenge" translated into both Python and Java. Each contains a one-line defect, along with passing and failing test cases.
[3] Big-Vul	Fan et al. (2020)	C/C++	264,919 functions (11,823 vulnerable) and 3,754 CVEs extracted from 348 projects.
[4] SVEN	He and Vechev (2023)	C/C++, Python	1,606 programs (803 pairs) of security fixes covering 9 critical CWEs from MITRE Top-25.
[5] SWE-bench	Jimenez et al. (2024)	Python	2,294 software engineering problems drawn from real GitHub issues and corresponding pull requests across 12 popular Python repositories.
[6] Codeforces	Community/Competitive (https://codeforces.com/)	Multilang	A competitive programming benchmark consisting of problems with human-submitted correct and buggy solutions, categorized by difficulty levels

URLs for datasets: [1] SVEN:<https://github.com/eth-sri/sven>; [2] Big-Vul:kaggle.com/datasets/kaggler10240/msr-data; [3] Defects4J:<https://github.com/rjust/defects4j>; [4] QuixBugs:<https://github.com/jkoppel/QuixBugs>; [5] SWE-bench:<https://github.com/SWE-bench/SWE-bench>; [6] Codeforces:<https://codeforces.com/problemset>

Fundamental limitations

- **Data Leakage and Limited Generalizability:** A primary limitation is the reliance on public benchmark datasets, which are susceptible to *data leakage*. According to Happe and Cito (2023) and Kang et al. (2023), popular benchmarks and security write-ups were likely included in the models' pre-training sets, which leads to overestimation of actual performance and creates a false sense of industrial readiness. This issue is further highlighted in Xia et al. (2024), which shows that models often leverage knowledge acquired during training when evaluating well-known open-source systems. These dependencies explain why many solutions struggle with *cross-project generalization* when applied to proprietary or unseen codebases, where the model cannot rely on memorized patterns. Li et al. (2023) explains that this difficulty arises because LLMs are often insensitive to nuances, where subtle differences between similar code sequences can lead to similar inferences regardless of whether the code is correct or erroneous.
- **High False Positive Rates and Limited Explainability:** A major technical barrier, particularly in tasks such as vulnerability detection, concerns the frequent generation of false positives, which undermines developer trust and increases validation overhead. Recent empirical evaluations (Ahmed et al., 2025; Sheng et al., 2024) consistently report that even state-of-the-art GenAI models exhibit limited predictive performance in realistic settings, with modest precision and F1-scores and a tendency to over-identify potential defects. This issue is compounded by limited statement-level localization and the generation of syntactically plausible yet logically inconsistent rationales, hindering developers' ability to understand and act upon model outputs. Furthermore, agent-based solutions for end-to-end task resolution (Ahmed et al., 2025; Yang et al., 2024) remain susceptible to error propagation and execution hallucinations, ultimately imposing substantial manual verification effort and limiting suitability for industrial-grade quality and security audits.
- **Contextual Isolation and Scale Limitations:** Current solutions are often restricted to analyzing small, localized code segments, failing to scale to the complexity of industrial systems. As identified by Kang et al. (2023), these tools struggle to track correlations between different parts of the code, especially when a defect depends on helper functions or configurations located in separate files. This isolation means the model is often "blind" to the broader system context, leading to syntactically correct results that are logically invalid for the specific project. Furthermore, as noted in Xia et al. (2024), many approaches depend on near-perfect technical documentation; in real-world legacy systems where such documentation is missing or outdated, the models struggle to understand the code's intended behavior, resulting in solutions that fail to address system-wide quality issues.

Challenges for adoption

- **High Barriers to Entry:** As detailed in RQ1d, a major challenge for adopting Open Models is the requirement for a robust local infrastructure, often including specialized and costly high-performance GPUs. This represents a significant financial and operational barrier; for instance, recent evaluations in our sample (e.g., Yang et al., 2024;

Ahmed et al., 2025) report the necessity of using high-end hardware such as NVIDIA A100 or H100 GPUs to achieve acceptable inference times.

- **Dependency on External Model Providers:** Beyond technical hurdles, many solutions in our sample rely on closed models accessed via APIs, which creates a significant limitation regarding operational stability and predictability. As documented by Kang et al. (2024), unannounced updates or changes in model versions (e.g., transitions between GPT-3.5 iterations) can alter output formats and break automated post-processing pipelines. Similarly, evaluations in Chen et al. (2024) and Ahmed et al. (2025) show that the effectiveness of these tools is often tied to specific high-end models, with performance degrading significantly when using different or smaller model versions. This instability poses a critical risk to the reproducibility of quality audits, as organizations lack control over the model's lifecycle and the consistency of results over time.
- **Complex Implementation and Lack of Usable Formats:** The solutions are generally not provided in ready-to-use formats. As described in RQ1c, they are typically distributed as code repositories or complex architectures (e.g., RAG) that demand specialized technical expertise for setup and replication. This lack of "out-of-the-box" usability is evident in studies like Jimenez et al. (2024), where specialized containerization (Docker) and environment configurations are required, creating a significant challenge for adoption by industry practitioners who require seamless integration into existing pipelines.
- **Absence of Formal Support and Maintenance:** Industrial-grade quality assurance requires long-term reliability. However, our analysis (RQ1g and RQ1i) reveals that nearly all identified solutions lack formal support channels, active communities, or maintenance plans. Most tools remain static prototypes as of their publication date, posing a significant risk for organizations that require constant security updates and stable support for their internal auditing tools.

Research Question 1f

What is the maturity level of these solutions?

The analysis reveals that the vast majority of the identified solutions remain in the early stages of technological maturity, primarily as prototypes developed and tested in academic or controlled settings. While most solutions fit this description, a few initiatives show progress toward practical adoption. For instance, the solution in Patel et al. (2024) underwent pilot testing and was validated across three companies. Another study (Ahmed et al., 2025) made its solution publicly available on the Hugging Face platform, allowing for initial dissemination⁹ and third-party experimentation. Despite these promising exceptions, the overall landscape is characterized by low technological maturity, with a general lack of solid evidence for the feasibility, scalability, and robustness of these solutions in industrial production environments.

Research Question 1g

Is there technical documentation, support, or an active community that facilitates its use in academic and industrial contexts?

⁹The solution had received 215 downloads in the 30 days preceding the study's conduct.

No clear support mechanisms, active community involvement, or long-term continuity plans were identified for the majority of the analyzed solutions. In general, the available technical documentation is limited and static, typically prepared by the developers solely at publication. While many solutions are available on platforms like GitHub, there is little evidence of initiatives to engage with users or communities outside the original academic environment. This lack of engagement hinders the adoption, evolution, and independent evaluation of these solutions in industrial contexts.

A single notable exception was identified, where the developers of one solution (Kessel, 2025) provided complementary information and updates through a Github.io¹⁰ page. This free service allows developers to publish user-friendly websites directly from their GitHub repositories, representing a significant step towards better documentation and user engagement. However, this appears to be an isolated case rather than a common practice.

Research Question 1h

What types of users or roles are supported by these solutions?

The analyzed solutions were found to target two distinct, highly technical user profiles, a focus that is a direct consequence of the tools' primary emphasis on source-code-level tasks rather than on systemic or architectural quality aspects. The main roles supported are:

- **Testing and QA Professionals:** The majority of the solutions are aimed at professionals in the testing and Quality Assurance (QA) areas (Chen et al., 2024; De Vito et al., 2023; Index.dev, 2024; Kang et al., 2023, 2024; Kessel, 2025; Li et al., 2023; Mansur et al., 2024; Patel et al., 2024; Phan et al., 2024; Xia et al., 2025, 2024; Yang et al., 2024; Yu et al., 2025). These tools are designed to reduce manual effort in test creation, increase test coverage, and accelerate verification and validation cycles.
- **Application Security (AppSec) Specialists:** A second group of solutions is specifically targeted at security specialists. These tools focus on the automated identification and, in some cases, exploitation of security vulnerabilities directly within source code snippets (Ahmed et al., 2025; Bhandari et al., 2025; Happe and Cito, 2023; Sedaily, 2024; Shao et al., 2025; Sheng et al., 2024; Steenhoek et al., 2025).

Notably, we found a scarcity of solutions aimed at roles concerned with more systemic and preventive quality areas, such as *Software Architects* (for identifying design flaws) or **Quality Managers** and *Reliability Engineers* (for overseeing formal processes like FMEA or FTA). This highlights that the current tool ecosystem is tailored for practitioners directly involved in code implementation and verification, rather than for those in more strategic or design-focused engineering roles.

Research Question 1i

What type of technical support is offered?

¹⁰ <https://github.io/>

A significant challenge for the adoption of the analyzed solutions is the general lack of formal technical support. For the most part, support is limited to static technical documentation provided within the code repositories. These materials, which typically cover basic installation, configuration, and usage guidelines, are often prepared only at publication and appear to lack ongoing maintenance. We observed a notable absence of standard industry support mechanisms such as dedicated forums, active technical communities, or formal support channels.

Despite this general lack, a few solutions demonstrated some level of openness to user interaction, albeit through informal channels. We identified one solution that published a contact link on its GitHub.io site (Kessel, 2025), another that provided a direct email address (Phan et al., 2024), and a third that offered a Discord¹¹ channel for communication (Happe and Cito, 2023). While these initiatives indicate a willingness among some developers to engage with users, they do not constitute the robust, reliable support infrastructure typically required for production adoption.

Research Question 1j

Is there collaboration with industry to validate the proposed solutions through case studies or experiments?

Our analysis reveals a near-total absence of direct collaboration between academic researchers and industry in the development and validation of the proposed solutions. The vast majority of solutions were developed exclusively in controlled settings, with no reported institutional partnerships or empirical validation in real-world industrial settings.

Two cases warrant specific mention as they approach, but do not fully represent, industrial collaboration. The only substantial exception identified was the solution presented in Patel et al. (2024), which was validated through a case study across three companies, demonstrating clear industry engagement. Another study (De Vito et al., 2023) involved an evaluation undertaken by recruited professionals. However, the study provided no evidence of an institutional link between these participants and the industrial sector, which prevents this case from being characterized as a formal industry collaboration.

Therefore, aside from one clear exception, the landscape is predominantly composed of solutions developed and validated in isolation from the environments in which they are intended to be used.

5 Discussion

5.1 Synthesis of findings and answer to RQ1

Our detailed analysis across the ten sub-questions (RQ1a-RQ1j) reveals a cohesive picture of the current landscape. While a growing number of solutions are becoming publicly accessible, primarily through GitHub repositories (RQ1a), they largely exist as academic prototypes with low technological maturity (RQ1f). These solutions employ a mix of powerful

¹¹ <https://discord.com/>

proprietary models, such as GPT-4, and open-source alternatives, such as Llama (RQ1b), but significant barriers hinder their practical adoption. Open-source models often require substantial local infrastructure, including high-performance GPUs (RQ1d), while both types demand complex configurations and specialized knowledge to be used effectively (RQ1c).

A critical and recurring theme is the profound lack of industrial validation. The vast majority of solutions are evaluated using synthetic or benchmark datasets, which limits the generalizability of their results to real-world industrial problems. This gap is exacerbated by a reliability gap, characterized by poor cross-project prediction and potential data leakage in existing benchmarks (RQ1e), as well as a near-total absence of collaboration with industry partners (RQ1j), and a scarcity of robust technical documentation, community support, or formal maintenance plans (RQ1g, RQ1i). Consequently, while these tools are primarily aimed at technical users such as testers and security specialists (RQ1h), they are not yet available in a ready-to-use format for industrial contexts. This synthesis leads to a consolidated answer to our main research question:

RQ1: What is the current state of industrial readiness and availability of Generative AI-based academic and commercial solutions to support software quality activities in Software Engineering?

The available Generative AI-based solutions for software quality constitute an emerging but immature ecosystem. The landscape is dominated by academically developed prototypes that, while conceptually promising, lack the technical maturity, industrial validation, and support infrastructure necessary for practical adoption. These are complemented by a smaller number of more mature commercial solutions that often lack public technical documentation, limiting their analysis. Overall, the field is in an early, experimental stage, with a significant gap between academic research and industry needs, driven by structural technical limitations in model precision and contextual awareness.

To provide a clear overview of the current state, Table 13 summarizes the primary benefits and limitations of the solutions analyzed in this assessment.

Table 13 Key benefits and limitations of the identified generative AI solutions for software quality. The two columns represent independent thematic summaries and are not directly correlated line by line

Key Benefits	Systemic Limitations
Automation of complex quality tasks.	Immaturity of Solutions: Most are academic prototypes, lacking robust support and are not production-ready.
Potential to reduce manual effort and accelerate verification cycles.	Barriers to Adoption: Solutions require significant hardware and specialized expertise, and reliance on proprietary APIs introduces risks (cost, privacy, lock-in).
Leverages state-of-the-art LLMs for nuanced analysis.	Lack of Industrial Grounding: Validation is predominantly on non-representative synthetic data, with a near-total absence of industry collaboration.
Ability to identify non-obvious issues through large-scale pattern analysis.	Intrinsic Technical Risks: Issues such as hallucinations, lack of explainability, and data leakage in benchmarks limit trust in critical environments.
Increasing accessibility of research artifacts through public repositories.	

5.2 Situating the findings: An immature but promising field

Our findings, while focused on a specific niche of software quality, strongly corroborate and provide concrete evidence for the broader perception that the application of Generative AI in Software Engineering remains in its early, experimental stages. Specifically, our empirical results regarding the generalization gap and lack of explainability align with recent theoretical concerns raised by Zhou et al. (2025) and Siavvas et al. (2024) regarding the limitations of LLMs for critical tasks. Broadly state-of-the-art works such as Zhang et al. (2024) and Schmid et al. (2025) have noted the conceptual nature of many LLM applications in SE. Our study amplifies this by detailing this immaturity specifically within the quality domain: we provide tangible evidence of a landscape dominated by academic prototypes (RQ1f) that are validated almost exclusively on synthetic or benchmark datasets (RQ1e), reinforcing the significant gap between academic research and practical adoption. The scarcity of industrial validation identified in our study (RQ1j) is a critical issue that warrants deeper discussion. We posit several interconnected causes for this phenomenon:

- **The “Last Mile” Problem in Academic Research:** A significant barrier to the industrial impact of academic research is what can be described as the “last mile problem”. This concept, drawn from other fields, symbolizes the inefficiencies in the final stage of delivering a product or service to the end-user (Li et al., 2025). In our context, it represents the gap between a functional academic prototype and a robust, usable tool ready for industrial adoption. Our findings indicate that this gap manifests in two critical dimensions. On the *operational front*, we identified a lack of logistical readiness, evidenced by the scarcity of documentation, user support, and easy-to-use installers. Simultaneously, on the *technical front*, many solutions exhibit characteristics that can limit their actual practicality. As evidenced by Kang et al. (2024) and Li et al. (2023), performance on public benchmarks is often inflated by data leakage; consequently, these prototypes demonstrate poor cross-project prediction, failing to generalize when applied to proprietary source code that was not known during training. Furthermore, the prevailing black-box nature of these models prevents users from understanding why a specific fragment was flagged or how to remediate the identified issue (Sheng et al., 2024). Consequently, bridging the “last mile” requires a holistic effort, improving engineering usability while simultaneously solving the algorithmic hurdles of generalizability and explainability.
- **The “Strategic Adoption Dilemma” for Industry:** Companies face a significant dilemma when deciding how to adopt and validate these solutions. On the one hand, using powerful, *Closed Models* (RQ1b) raises concerns about data privacy and security, as well as unpredictable costs. This approach introduces a substantial risk of *vendor lock-in*, a well-documented challenge in cloud computing where organizations become dependent on a single provider’s proprietary technology, making future migration difficult and costly (Opara-Martins et al., 2016). This risk is particularly acute with LLM APIs, which often lack the predictable update cycles and formal service-level agreements of traditional enterprise software. On the other hand, using *Open Models* requires substantial upfront investment in specialized hardware (RQ1d). Our finding that only NVIDIA GPUs were mentioned directly reflects this challenge; the maturity of NVIDIA’s CUDA software platform has created a powerful ecosystem lock-in, making alternatives from

other vendors practically unviable for most researchers seeking stability and performance (Cusumano, 2023; Manganelli, 2025). Compounding this dilemma, both approaches carry the inherent risk of relying on models trained on vast, uncurated datasets that may introduce biases or security flaws. This complex web of trade-offs significantly slows down large-scale industrial experimentation and validation.

- **Scarcity of Realistic Public Data:** A fundamental challenge in empirical software engineering is that high-quality industrial data, such as comprehensive defect reports or real-world security vulnerabilities, is often proprietary and highly sensitive. This scarcity forces many researchers to rely on publicly available benchmarks (as shown in Table 12) or to use synthetic data, which are artificially generated datasets designed to emulate original data without revealing sensitive observations (Wang et al., 2024). While these approaches are helpful for controlled experimentation, they may not accurately reflect the complexity and scale of industrial software systems. This creates a problematic feedback loop in which academic solutions are not trained on or validated with industrial-grade data, further widening the gap between research and practice.

Furthermore, our analysis of the grey literature revealed a strong “academic echo,” with a high concentration of preprints from repositories like arXiv¹². The limited presence of industrial white papers, technical blogs, or open-source projects from industry actors reinforces the perception that widespread, practical adoption is still in its nascent stages, with much of the current knowledge being generated and disseminated by academic actors.

6 Threats to validity

In this section, we discuss the potential threats to the validity of this study that may compromise the reliability and applicability of the results obtained. To ensure the robustness of the findings, it is essential to identify, discuss, and, whenever possible, minimize these impacts (Verdecchia et al., 2023). Furthermore, ensuring the validity of the results is crucial to guarantee their applicability to the target population (Travassos et al., 2002; Wohlin et al., 2000). In this study, we analyzed four main categories of threats that could contribute to future replication attempts of the RMLR: *internal validity*, *external validity*, *construct validity*, and *conclusion validity* (Shull et al., 2008). We present the strategies adopted to minimize their impacts and highlighted possible limitations that should be considered when interpreting the results.

6.1 Internal validity

- **Threats:** The “rapid” nature of the RMLR implied that no in-depth assessment of the methodological quality of the included studies was conducted. However, it is a common practice in the execution of RRs, which aim to deliver results in a shorter timeframe, even if this implies simplifying traditional steps. As a consequence, technologies developed and evaluated with lower methodological rigor may have been included and treated with the same weight as those with more robust methodologies. Furthermore,

¹² <https://arxiv.org/>

the inclusion of grey literature introduces sources that have not undergone peer review, whose content may reflect commercial bias or opinions unsupported by solid empirical evidence.

- **Mitigations:** To mitigate these effects, we acknowledge these limitations as inherent to the RMLR approach. Our primary mitigation strategy was to design the research questions themselves to act as a proxy for quality assessment. Specifically, subquestions regarding maturity (RQ1f), support (RQ1g), and industrial collaboration (RQ1j) allowed us to indirectly assess the practical relevance and robustness of each solution's evidence. For the grey literature, we adopted a source diversification strategy, consulting multiple channels (Google, GitHub, and PapersWithCode) to expand coverage and minimize platform-specific biases.

6.2 External validity

- **Threats:** Threats to external validity concern the generalizability of our findings. The scope of our search was deliberately limited, which introduces four main considerations. First, the search in the scientific literature was restricted to the Scopus database. Second, the grey literature analysis considered only the first 100 results from each queried platform, potentially excluding relevant sources ranked lower by the search engine. Third, the study included only articles published in English after December 2021, which excludes solutions published in other languages or earlier periods. Finally, a temporal threat exists due to the rapid evolution of Generative AI, suggesting that new tools or versions may have emerged between our data collection (April-May 2025) and publication.
- **Mitigations:** These limitations were deliberate scoping decisions, necessary to conduct a "rapid" assessment focused on the most recent and accessible solutions. This approach was justified in the Methodology section, where we emphasized that the objective was not to be exhaustive but to provide a structured overview of current practice. Regarding the temporal threat, we argue that the critical findings of this assessment, specifically the "Last Mile Problem" and the lack of industrial readiness, are systemic characteristics of the field rather than transient features of specific tools. Therefore, the structural diagnostic presented here remains a valid and necessary baseline for understanding the persistent gap between academic prototypes and industrial needs.

6.3 Construct Validity

- **Threats:** The main threat in this category is related to the operationalization of the search strings. The concept of "software quality" is broad and multifaceted, making it challenging to ensure that all its dimensions are considered when formulating queries. Furthermore, other terms used, such as those related to Generative AI technologies, the types of expected solutions, or the application context, may also be ambiguous or inconsistently used across sources, especially in the grey literature. Additionally, the need to shorten the search string for Google, imposed by the search engine's technical limitations, may have compromised the consistency and scope of data collection across

the different sources.

- **Mitigations:** To mitigate these threats, we employed a diversified strategy. First, we used the PICOC approach to systematically structure and decompose our research question into specific, searchable terms. This process, which included the use of wildcards (*) in Scopus, helped capture terminological variations. Second, the strings were iteratively refined through iterative refinement through pilot searches and collaborative discussions among the authors. Finally, to mitigate the risk of losing sources from the simplified Google query, our iterative approach included a complementary search step. During the analysis, we also assessed the relevant hyperlinks on the pages initially retrieved, allowing us to expand coverage beyond the query's direct results. While the reduction of the search string for Google remains a limitation, we argue that this combination of a structured approach (PICOC) and an exploratory approach (link assessment) provided a robust and transparent data-collection process.

6.4 Conclusion validity

- **Threats:** Researcher bias in the selection and classification of studies is an inherent threat to any review, especially when it involves subjective judgments. The interpretation of inclusion and exclusion criteria, as well as the categorization of findings, may vary among team members, potentially affecting the consistency of the results.
- **Mitigations:** To mitigate this threat, we implemented a rigorous, multi-stage review protocol throughout the screening process. For the Scopus literature, each stage (title, abstract, and full text) involved independent evaluations by at least 2 authors. Disagreements were systematically resolved either through a predefined consensus rule, a third-party mediator, or a final consensus meeting, as detailed in Section 3.3. For the grey literature, the sources were initially distributed among three authors for independent readings. This was followed by a cross-review stage, in which a second author reassessed each source, ensuring that each item underwent a dual review before discrepancies were discussed in a consensus meeting. This structured, multi-reviewer approach, supported by tools like JabRef for managing scientific literature, was designed to ensure greater consistency and reliability in the final selection of publications, thereby strengthening the validity of our conclusions.

7 Conclusion

This study aimed to assess the industrial readiness and availability of Generative AI-based solutions currently available to support quality-related activities in Software Engineering. Our findings reveal a field in its early stages, characterized by a vibrant academic interest that has yet to translate into a mature ecosystem of industry-ready tools. The landscape is marked by a significant gap between the state of the art in academic research and the state of practice in industry, primarily due to a lack of industrial validation, high adoption barriers, and structural technical limitations regarding model precision and generalizability.

7.1 Implications of this work

The results of this study carry significant implications for both researchers and practitioners. *For researchers*, our findings highlight a critical need to bridge the “last mile” gap between algorithmic prototypes and usable solutions. Future work should prioritize industrial collaborations, address the reliability gap through improved explainability and context-awareness, and validate on real-world data rather than relying solely on synthetic benchmarks, and create well-documented, reproducible artifacts. *For practitioners*, this work serves as a pragmatic guide to the current landscape, underscoring that Generative AI should be adopted as an assistive technology that augments human expertise, not as a fully autonomous replacement. Practitioners must carefully consider the “strategic adoption dilemma,” balancing the costs and risks of proprietary models against the infrastructure and expertise demands of open-source alternatives.

7.2 Limitations of this study

While we employed a rigorous methodology, this study has inherent limitations that should be considered. First, our assessment relies on the reported evidence, documentation, and repository data of the identified solutions; we did not perform a hands-on execution or benchmark testing of the tools themselves. Second, by focusing on publicly documented solutions, our search may have missed proprietary, in-house tools used silently within specific companies. Additionally, while the use of Scopus as the primary academic database provided a broad baseline, it is possible that some specific tool-related papers indexed in other databases were not captured, a risk we mitigated by triangulating with extensive grey literature searches. Finally, regarding the rapid evolution of Generative AI, we acknowledge that individual tool versions may have changed since our data collection. However, as discussed in Section 5, we contend that the identified structural challenges, such as the “Last Mile Problem”, are systemic and persist independently of short-term model updates.

7.3 A roadmap for future research

Based on the gaps and opportunities identified, we propose a clear roadmap to advance the field towards higher quality standards:

1. **Industrial Case Studies and Benchmarks:** There is an urgent need for in-depth case studies evaluating academic prototypes in real-world industrial settings. Furthermore, although challenging due to confidentiality constraints and the limited public availability of proprietary systems, the community would greatly benefit from the development of standardized benchmarks grounded in realistic industrial scenarios. Possible paths forward include anonymized datasets, curated subsets of proprietary systems under controlled collaboration agreements, or systematically constructed benchmarks derived from representative open-source repositories (e.g., SWE-bench Jimenez et al., 2024). Such initiatives would enhance the ecological validity, fairness, and reproducibility of evaluations while strengthening the bridge between academic research and industrial practice.

2. **Quality Guidelines and Standardization:** Future research must move beyond ad-hoc experimentation to establish quality models for GenAI-based SE solutions. This includes defining specific standards for robustness, explainability, and integration readiness. Furthermore, there is a critical need for systematic criteria to guide model selection, helping practitioners evaluate the trade-offs between closed and open architectures based on specific industrial constraints.
3. **Expanding the Scope Across the SDLC and ISO 25010:** As our mapping revealed a heavy concentration of solutions in the *Coding* and *Testing* phases, future research must expand to earlier stages of the SDLC. There is a significant opportunity to explore GenAI solutions in *Requirements* engineering and *Design* architecture. Furthermore, the field must move beyond isolated code-level fixes (primarily targeting Security and Reliability) to address the full spectrum of systemic quality attributes defined by the ISO/IEC 25010 standard, such as usability, performance efficiency, and portability.
4. **Addressing the “Last Mile” Problem:** Research is needed on the software engineering challenges of transforming GenAI prototypes into robust, industry-ready tools. This requires a dual approach: (i) addressing operational usability through IDE integration, human-computer interaction for AI assistants, and seamless CI/CD deployment; and (ii) solving fundamental reliability hurdles, specifically by enhancing cross-project generalization and providing transparent, actionable rationales for AI-generated outputs.

In conclusion, closer, more structured collaboration between academia and industry will be essential to realize the potential of Generative AI and deliver reliable, scalable, and effective solutions that can be integrated into the daily operations of software quality assurance.

Acknowledgements We want to thank the research team for their dedication and collaborative efforts throughout the development of this study. Prof. Travassos is a CNPq Researcher and CNE FAPERJ.

Author Contributions A.G., P.A.G., and C.H.B. led the execution and writing of this study. R.M.M. and G.H.T. supervised, reviewed the text, and collaborated on the study’s feasibility. S.R. and R.F. assisted with the text review.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614).

Data Availability We provide an archive of our artifacts into Zenodo database under accession number 17096048 and are publicly available at the following URL: <https://zenodo.org/records/17096048>

Declarations

Competing interests The authors declare no competing interests.

Ethics approval and consent to participate Not applicable.

Consent for publication All the participants were informed and freely signed a consent form to participate in the research.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the

article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Ahmad, Z., & Kuo, M. M. Y. (2025). Improving story points estimation using ensemble machine learning. *Software Quality Journal*, 33(1), 35. <https://doi.org/10.1007/s11219-025-09731-6>
- Ahmed, M. B. U., Harzevili, N. S., Shin, J., Pham, H. V., & Wang, S. (2025). *SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection*. Preprint at [arxiv:2505.19828](https://arxiv.org/abs/2505.19828)
- Alagarsamy, S., Tantithamthavorn, C., Takerngsaksiri, W., Arora, C., & Aleti, A. (2025). Enhancing large language models for text-to-testcase generation. *Journal of Systems and Software*, 230, 112531. <https://doi.org/10.1016/j.jss.2025.112531>
- Baldassarre, M. T., Caivano, D., Dimauro, G., Romano, S., & Scanniello, G. (2021). On internet-of-things devices in ambient assisted living solutions. In *International Conference on Information Systems Development*.
- Banh, L., Holldack, F., & Strobel, G. (2025). Copiloting the future: How generative ai transforms software engineering. *Information and Software Technology*, 183, 107751. <https://doi.org/10.1016/j.infsof.2025.107751>
- Bazzan, T., Olojo, B., Majda, P., Kelly, T., Yilmaz, M., Marks, G., & Clarke, P. (2024). Analysing the role of generative AI in software engineering - results from an MLR. In *Systems, Software and Services Process Improvement. EuroSPI 2024* (pp. 163–180). https://doi.org/10.1007/978-3-031-71139-8_11
- Bhandari, G., Gavric, N., & Shalaginov, A. (2025). Generating vulnerability security fixes with code language models. *Information and Software Technology*, 185, 107786. <https://doi.org/10.1016/j.infsof.2025.107786>
- Cartaxo, B., Pinto, G., & Soares, S. (2018). The role of rapid reviews in supporting decision-making in software engineering practice. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018 (EASE '18)* (pp. 24–34). Association for Computing Machinery (ACM), Christchurch, New Zealand. 10.1145/3210459.3210462
- Chen, X., Lin, M., Schärli, N., & Zhou, D. (2024). Teaching large language models to self-debug. In *12th International Conference on Learning Representations (ICLR 2024)*.
- Cruzes, D. S., & Dyba, T. (2011). Recommended steps for thematic synthesis in software engineering. In *2011 International Symposium on Empirical Software Engineering and Measurement* (pp. 275–284). IEEE, Banff, Canada. <https://doi.org/10.1109/ESEM.2011.36>
- Cusumano, M. (2023). Nvidia at the center of the generative ai ecosystem—for now. *Communications of the ACM*, 67(1), 33–35.
- De Vito, G., Lambiase, S., Palomba, F., & Ferrucci, F. (2023). Meet C4SE: Your new collaborator for software engineering tasks. In *2023 49th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 235–238). IEEE, Dures, Albania. <https://doi.org/10.1109/SEAA60479.2023.00044>
- Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., & Lahiri, S. (2024). Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 50, 2254–2268. <https://doi.org/10.1109/TSE.2024.3428972>
- Fan, J., Li, Y., Wang, S., & Nguyen, T. N. (2020). A c/c++ code vulnerability dataset with code changes and cve summaries. In *Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20)* (pp. 508–512). ACM. <https://doi.org/10.1145/3379597.3387501>
- Garousi, V., Felderer, M., Mäntylä, M. V., & Rainer, A. (2020). Benefitting from the grey literature in software engineering research. In M. Felderer, & G. H. Travassos (Eds.), *Contemporary Empirical Methods in Software Engineering* (pp. 385–413). Springer, Cham. https://doi.org/10.1007/978-3-030-32489-6_14
- Garousi, V., Felderer, M., & Mäntylä, M. V. (2016). The need for multivocal literature reviews in software engineering: complementing systematic literature reviews with grey literature. In *Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering (EASE '16)* (p. 26). Association for Computing Machinery (ACM), Limerick, Ireland. <https://doi.org/10.1145/291597.0.2916008>
- Garousi, V., Joy, N., Jafarov, Z., Keleş, A. B., Değirmenci, S., Özdemir, E., & Zarringhalami, R. (2025). *AI-powered software testing tools: A systematic review and empirical assessment of their features and limitations*. Preprint at <https://arxiv.org/abs/2409.00411>

- Google (2025). *Google Programmable Search Engine*. <https://programmablesearchengine.google.com/about/>
- Hamel, C., Michaud, A., Thuku, M., Skidmore, B., Stevens, A., Nussbaumer-Streit, B., & Garritty, C. (2021). Defining rapid reviews: a systematic scoping review and thematic analysis of definitions and defining characteristics of rapid reviews. *Journal of Clinical Epidemiology*, 129, 74–85. <https://doi.org/10.1016/j.jclinepi.2020.09.041>
- Happe, A., & Cito, J. (2023). Getting pwn'd by AI: Penetration testing with large language models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2023)* (pp. 2082–2086). Association for Computing Machinery (ACM), San Francisco, CA, USA. <https://doi.org/10.1145/3611643.3613083>
- He, J., & Vechev, M. (2023). Large language models for code: Security hardening and adversarial testing. In *ACM CCS*. <https://doi.org/10.1145/3576915.3623175>
- Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 220. <https://doi.org/10.1145/3695988>
- Index.dev (2024). *How GenAI Transforms Software Development in 11 Ways*. <https://www.index.dev/blog/11-generative-ai-use-cases-software-development>
- ISO/IEC (2011). Systems and software engineering – systems and software quality requirements and evaluation (SQuaRE) – system and software quality models. Standard ISO/IEC 25010:2011, International Organization for Standardization.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=VT9ovN-cfF>
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA '14)* (pp. 437–440). ACM. <https://doi.org/10.1145/2610384.2628055>
- Kalouptoglou, I., Siavvas, M., Ampatzoglou, A., Kehagias, D., & Chatzigeorgiou, A. (2025). Transfer learning for software vulnerability prediction using transformer models. *Journal of Systems and Software*, 227, 112448. <https://doi.org/10.1016/j.jss.2025.112448>
- Kalouptoglou, I., Siavvas, M., Ampatzoglou, A., Kehagias, D., & Chatzigeorgiou, A. (2026). Locvul: Line-level vulnerability localization based on a sequence-to-sequence approach. *Information and Software Technology*, 189, 107940. <https://doi.org/10.1016/j.infsof.2025.107940>
- Kang, S., Yoon, J., & Yoo, S. (2023). Large language models are few-shot testers: Exploring LLM-based general bug reproduction. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)* (pp. 2312–2323). IEEE Press, Melbourne, Victoria, Australia. <https://doi.org/10.1109/ICSE48619.2023.00194>
- Kang, S., Yoon, J., Askarbekkyzy, N., & Yoo, S. (2024). Evaluating diverse large language models for automatic and general bug reproduction. *IEEE Transactions on Software Engineering*, 50(10), 2677–2694. <https://doi.org/10.1109/TSE.2024.3450837>
- Kessel, M. (2025). *Test-driven Software Experimentation with LASSO: an LLM Prompt Benchmarking Example*. Preprint at <https://arxiv.org/abs/2410.08911>
- Kikuta, D., Ikeuchi, H., & Tajiri, K. (2025). *ChaosEater: Fully Automating Chaos Engineering with Large Language Models*. Preprint at [arxiv:2501.11107](https://arxiv.org/abs/2501.11107)
- Kitchenham, B. (2004). Procedures for performing systematic reviews. Technical Report TR/SE-0401 and NICTA Technical Report 0400011T.1, Software Engineering Group, Department of Computer Science, Keele University and Empirical Software Engineering, National ICT Australia Ltd., Keele, UK and NSW, Australia. Joint Technical Report.
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174. <https://doi.org/10.2307/2529310>
- Li, H., Wei, D., Moffitt, K. C., & Vasarhelyi, M. A. (2025). Addressing the last mile problem in open government data: Using AIS technologies to enhance governmental financial reporting. *Journal of Emerging Technologies in Accounting*. Forthcoming. <https://doi.org/10.2308/JETA-2024-018>
- Li, T.-O., Zong, W., Wang, Y., Tian, H., Wang, Y., Cheung, S.-C., & Kramer, J. (2023). Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)* (pp. 14–26). <https://doi.org/10.1109/ASE56229.2023.00089>
- Lin, D., Koppel, J., Chen, A., & Solar-Lezama, A. (2017). Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion '17)* (pp. 55–56). ACM. <https://doi.org/10.1145/3135932.3135941>

- Manganelli, A. (2025). Foundation models and generative ai applications: what competitive concerns? Available at SSRN 5242028. <https://doi.org/10.1080/17441056.2026.2641378>
- Mansur, E., Chen, J., Raza, M. A., & Wardat, M. (2024). RAGFix: Enhancing LLM code repair using RAG and Stack Overflow posts. In *2024 IEEE International Conference on Big Data (BigData)* (pp. 7491–7496). IEEE, Washington, D.C., USA. <https://doi.org/10.1109/BigData62323.2024.10825785>
- Opara-Martins, J., Sahandi, R., & Tian, F. (2016). Critical analysis of vendor lock-in and its impact on cloud computing migration: a business perspective. *Journal of Cloud Computing: Advances, Systems and Applications*, 5(4). <https://doi.org/10.1186/s13677-016-0054-z>
- Patel, S., Patil, K., & Chumchu, P. (2024). BHRAMARI: Bug driven highly reusable automated model for automated test bed generation and integration. *Software Impacts*, 21, 100687. <https://doi.org/10.1016/j.simpa.2024.100687>
- Pena, C., Cartaxo, B., Steinmacher, I., Badampudi, D., Silva, D., Ferreira, W., Almeida, A., Kamei, F. K., & Soares, S. (2024). Comparing the efficacy of rapid review with a systematic review in the software engineering field. *Journal of Software: Evolution and Process*, 37. <https://doi.org/10.1002/smr.2748>
- Petticrew, M., & Roberts, H. (2006). *Systematic Reviews in the Social Sciences: A Practical Guide*. Malden: Blackwell Publishing.
- Phan, H. N., Nguyen, P. X., & Bui, N. D. (2024). Hyperagent: Generalist software engineering agents to solve coding tasks at scale. arXiv preprint [arXiv:2406.11912](https://arxiv.org/abs/2406.11912)
- Pizard, S., Lezama, J., García, R., Vallespir, D., & Kitchenham, B. (2024). Using rapid reviews to support software engineering practice: a systematic review and a replication study. *Empirical Software Engineering*, 30(1). <https://doi.org/10.1007/s10664-024-10545-6>
- Qin, Y., Wang, S., Lou, Y., Dong, J., Wang, K., & Li, X. (2025). Soapfl: A standard operating procedure for llm-based method-level fault localization. *IEEE Transactions on Software Engineering*, 51(4), 1173–1187. <https://doi.org/10.1109/TSE.2025.3543187>
- Rafi, S., Akbar, M. A., & Khan, A. A. (2025). Assessing software product quality in devops: An iso 25010:2023 perspective. In *Proceedings of the 2025 Evaluation and Assessment in Software Engineering (EASE Companion '25)*. ACM, New York, NY, USA. <https://doi.org/10.1145/3727967.3756847>
- Raghi, K. R., Sudha, K. M., S. A., Joshua, S., & S. (2024). Software development automation using generative AI. In *2024 International Conference on Emerging Research in Computational Science (ICERCS)* (pp. 1–6. IEEE). <https://doi.org/10.1109/ICERCS63125.2024.10894980>
- Rocha, S., Feitosa, R., Galeno, L., & Travassos, G. H. (2025). A metaprotocol for a family of rapid multivocal reviews of generative ai in the software industry. In *XXXIX Simpósio Brasileiro de Engenharia de Software* (pp. 804–810). <https://doi.org/10.5753/sbes.2025.11577>
- Russo, D. (2024). Navigating the complexity of generative ai adoption in software engineering. *ACM Transactions on Software Engineering and Methodology*, 33, 50. <https://doi.org/10.1145/3652154>
- Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., Liu, H., & Koziolk, A. (2025). *Software Architecture Meets LLMs: A Systematic Literature Review*. Preprint at [arxiv:2505.16697](https://arxiv.org/abs/2505.16697)
- SEDaily (2024). Weaving Generative AI into DevSecOps. <https://softwareengineeringdaily.com/2024/04/18/gitlab-weaving-generative-ai-devsecops/>
- Shao, M., Xi, H., Rani, N., Udeshi, M., Putrevu, V. S. C., Milner, K., Dolan-Gavitt, B., Shukla, S. K., Krishnamurthy, P., Khorrami, F., Karri, R., & Shafique, M. (2025). *CRAKEN: Cybersecurity LLM Agent with Knowledge-Based Execution*. Preprint at [arxiv:2505.17107](https://arxiv.org/abs/2505.17107)
- Sheng, Z., Wu, F., Zuo, X., Li, C., Qiao, Y., & Lei, H. (2024). Research on the LLM-driven vulnerability detection system using LProtector. In *2024 IEEE 4th International Conference on Data Science and Computer Application (ICDSCA)* (pp. 192–196). IEEE, Guilin, China. <https://doi.org/10.1109/ICDSCA63855.2024.10859408>
- Shull, F., Singer, J., & Sjöberg, D. I. K. (Eds.) (2008). *Guide to Advanced Empirical Software Engineering*. Springer, London, UK. <https://doi.org/10.1007/978-1-84800-044-5>
- Siavvas, M., Kalouptsoglou, I., Gelenbe, E., Kehagias, D., & Tzovaras, D. (2024). Transforming the field of vulnerability prediction: Are large language models the key? In *Proceedings of the 32nd International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS 2024)*. <https://doi.org/10.1109/MASCOTS64422.2024.10786575>
- Steenhoek, B., Rahman, M. M., Roy, M. K., Alam, M. S., Tong, H., Das, S., Barr, E. T., & Le, W. (2025). To Err is Machine: Vulnerability Detection Challenges LLM Reasoning. Preprint at <https://arxiv.org/abs/2403.17218>
- Tang, Y., Liu, Z., Zhou, Z., & Luo, X. (2024). Chatgpt vs sbst: A comparative assessment of unit test suite generation. *IEEE Transactions on Software Engineering*, 50, 1340–1359. <https://doi.org/10.1109/TSE.2024.3382365>
- Terragni, V., Vella, A., Roop, P., & Blincoe, K. (2025). The future of AI-driven software engineering. *ACM Transactions on Software Engineering and Methodology*, 34(5), 120. <https://doi.org/10.1145/3715003>

- Travassos, G. H., Gurov, D., & Amaral, E. A. G. d. (2002). Introdução à engenharia de software experimental. Relatório Técnico RT-ES-590/02, Programa de Engenharia de Sistemas e Computação, COPPE/UFRJ, Rio de Janeiro, Brazil. <https://pesc.coppe.ufrj.br/index.php/pt-BR/publicacoes-pesquisa/details/15/589>
- Travassos, G. H., Rocha, S., Feitosa, R., Assis, F., Gonçalves, P., Gheventer, A., Galeno, L., Sasse, A., Guimarães, J. C., Brito, C., & Wieland, J. P. (2025). Lessons learned from the use of generative ai in engineering and quality assurance of a web system for healthcare. In *Proceedings of the XXIII Brazilian Symposium on Software Quality (SBQS '25)*. SBC, São José dos Campos, SP, Brazil. <https://doi.org/10.5753/sbqs.2025.15015>
- Verdecchia, R., Engström, E., Lago, P., Runeson, P., & Song, Q. (2023). Threats to validity in software engineering research: A critical reflection. *Information and Software Technology*, 164, 107329. <https://doi.org/10.1016/j.infsof.2023.107329>
- Wang, P., Loignon, A. C., Shrestha, S., Banks, G. C., & Oswald, F. L. (2025). Advancing organizational science through synthetic data: A path to enhanced data sharing and collaboration. *Journal of Business and Psychology* 40, 771–797. <https://doi.org/10.1007/s10869-024-09997-w>. Published online: 06 December 2024
- Washizaki, H. (Ed.) (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide), Version 4.0*. IEEE Computer Society, Waseda University, Japan. <https://www.swebok.org>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2000). *Experimentation in Software Engineering: an Introduction*. USA: Kluwer Academic Publishers.
- Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2025). Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), 037. <https://doi.org/10.1145/3715754>
- Xia, C. S., Paltenghi, M., Le Tian, J., Pradel, M., & Zhang, L. (2024). Fuzz4All: Universal fuzzing with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (p. 126). Association for Computing Machinery (ACM), Lisbon, Portugal. <https://doi.org/10.1145/3597503.3639121>
- Xu, H., Liu, H., Wu, Y., Kang, X., Chen, X., & Liu, Y. (2026). Exploring the potential and limitations of large language models for novice program fault localization. *Journal of Systems and Software*, 234, 112731. <https://doi.org/10.1016/j.jss.2025.112731>
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. R., & Press, O. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://doi.org/10.52202/079017-1601>
- Yang, J. (2025). *Thematic Coding in Qualitative Research: A Practical Guide for Real Insights*. <https://www.usercall.co/post/thematic-coding-in-qualitative-research-a-practical-guide-for-real-insights>
- Yu, Z., Zhang, H., Zhao, Y., Huang, H., Yao, M., Ding, K., & Zhao, J. (2025). OrcaLoca: An LLM Agent Framework for Software Issue Localization. Preprint at [arxiv:2502.00350](https://arxiv.org/abs/2502.00350)
- Zhang, Z., Chen, C., Liu, B., Liao, C., Gong, Z., Yu, H., Li, J., & Wang, R. (2024). Unifying the Perspectives of NLP and Software Engineering: A Survey on Language Models for Code. Preprint at [arxiv:2311.07989](https://arxiv.org/abs/2311.07989)
- Zhou, X., Cao, S., Sun, X., & Lo, D. (2025). Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3708522>



André Gheventer is a PhD student in Computer Science at the Federal University of Rio de Janeiro (UFRJ), Brazil. His research focuses on the intersection of secure software engineering, privacy, and artificial intelligence. His specific interests include generative AI, threat analysis, and intrusion detection, particularly within automotive security operations. Contact: gheventer@ufrj.br.



Patrícia do Amaral Gurgel is a PhD student in Computer Science at the Federal University of Rio de Janeiro (UFRJ), Brazil. Her research focuses on the intersection of Experimental Software Engineering, Software Quality, and Information Security. Her specific interests include the application of generative AI for threat modeling, security requirements, and software testing. She has over 15 years of experience in Software Engineering and currently serves as a Systems Analyst at the Information Security Division at UFRJ. Contact: patriciaamaralgurgel@ufrj.br.



Carlos Brito is a Software Developer at CERN. He is currently pursuing his bachelor's degree in Computer Engineering at the Federal University of Rio de Janeiro (UFRJ), Brazil. Website: <https://carloshenriquefbf.github.io/>.



Rafael de Mello is an Associate Professor at the Federal University of Rio de Janeiro (UFRJ), where he leads the ITER Research Group. His research interests include Artificial Intelligence for Software Engineering, socio-technical aspects of software engineering, and empirical software engineering. He holds a Doctor of Science (D.Sc.) in Systems Engineering and Computer Science from COPPE/UFRJ. For inquiries, he can be contacted at rafaelmello@ic.ufrj.br. More information is available at: <https://rmaiani.github.io/rafaeldemello/>.



Sabrina Rocha de Souza is a PhD candidate in Systems and Computer Engineering at the Federal University of Rio de Janeiro, under a cotutelle agreement with the University Polytechnique Hauts-de-France (UPHF) in Computer Science. Her research interests include Software Engineering. Contact: sabrinarocha@cos.ufrj.br.



Rodrigo Feitosa is a PhD candidate in Systems and Computer Engineering at the Federal University of Rio de Janeiro, PESC/COPPE/UFRJ. His research interests include Software Engineering, with a focus on psychological safety, socio-technical factors, and requirements engineering. Contact: rfeitosa@cos.ufrj.br.



Guilherme Horta Travassos is a Full Professor of Software Engineering in the Systems Engineering and Computing Graduate Program at Coppe, Federal University of Rio de Janeiro. He is also a CNPq Researcher and CNE FAPERJ. His research interests include the engineering of contemporary software systems and experimental software engineering. He is a member of SBC, ACM, and ISERN. Website: <https://pesc.coppe.ufrj.br/~ght/>.

Authors and Affiliations

Andre Gheventer¹ · Patricia do Amaral Gurgel¹ · Carlos Henrique Brito² · Rafael Maiani de Mello¹ · Sabrina Rocha³ · Rodrigo Feitosa³ · Guilherme Horta Travassos³

✉ Andre Gheventer
gheventer@ufrj.br

Patricia do Amaral Gurgel
patriciaamaralgurgel@ufrj.br

Carlos Henrique Brito
carloshenriquefbf@poli.ufrj.br

Rafael Maiani de Mello
rafaelmello@ic.ufrj.br

Sabrina Rocha
sabrinarocha@cos.ufrj.br

Rodrigo Feitosa
rfeitosa@cos.ufrj.br

Guilherme Horta Travassos
ght@cos.ufrj.br

¹ Computing Institute, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil

² Polytechnic School of Engineering, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil

³ Systems Engineering and Computer Science Graduate Program, Federal University of Rio de Janeiro, Rio de Janeiro, Brazil